1

第1章 Spring Boot と開発環境の概要

章の目標

- フレームワークと Spring Framework について理解する
- Spring Boot の基本概念と特徴を把握する
- Maven を用いてライブラリのインストールができる

1

第1章 Spring Boot と開発環境の概要

章の目標

- フレームワークとSpring Framework について理解する
- Spring Boot の基本概念と特徴を把握する
- Maven を用いてライブラリのインストールができる

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスから B クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンを A クラスに渡す。というイメージに近いです。
A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring は MVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（View）、データの処理（Model）、そしてユーザの入力と画面表示の制御（Controller）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章から MVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring は Java の Web アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年 11 月リリース)を利用します。

Spring の特徴

1. DI（依存性注入）

Spring では、「DI」という概念を使って、異なるクラス間の繋がり(依存関係)を効率よく管理できます。A クラスからB クラスを直接呼び出すのではなく、B クラスと繋がるリモコンを予め倉庫に入れておいて、必要な時にそのリモコンをA クラスに渡す。というイメージに近いです。
A クラスは渡されたリモコンの「スイッチ（メソッド）」を押すだけで機能を使えます。

この仕組みのおかげで、**よく使われるクラスを簡単に呼び出し、プログラムの柔軟性と再利用性を高める**ことができます。DI については、3 章でご説明します。

2. MVC モデルの設計理念

Spring はMVC（Model-View-Controller）モデルを基にしたアプリケーション開発をサポートします。これにより、画面の表示（**View**）、データの処理（**Model**）、そしてユーザの入力と画面表示の制御（**Controller**）といったプログラムの役割を明確に分けることができ、より管理しやすいコードを作成できます。2 章からMVC パターンを活用した実装の仕方を学習します。

3. ライブラリの統合

Spring は他の多くのフレームワークやライブラリと組み合わせて利用することが可能です。例えば、ビューテンプレートに「Thymeleaf」、データベースアクセスに「Hibernate」、ログ出力に「Apache Log4j」といった様々なツールを組み合わせることができます。次の節で詳しくご説明します。

以上の特徴により、Spring はJava のWeb アプリケーション開発をより効率的かつ柔軟にするための強力なツールとなっています。

本書では、Spring Framework 6.0(2022 年11 月リリース)を利用します。

要素	JDBC	Spring Data JPA
外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。
ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。
冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。
例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時に SQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。
リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。

Spring Data JPA は、よりシンプルで安全、そして効率的なデータアクセスを提供することで、データベースプログラミングを容易にします。

要素	JDBC	Spring Data JPA
外部ライブラリの依存	JDBC ドライバー(mysql-connector-j.jar)を手動でプロジェクトに追加する必要がある。ダウンロードやビルドパスの設定などの追加作業が伴う。	Maven で必要なライブラリが自動的に管理される。手動での追加作業は不要。
ハードコーディング	接続情報（URL、ユーザー名、パスワード）がソースコード内に直接書かれることが多く、セキュリティリスクになり得る。環境の変更があるたびにコードの変更が必要。	接続情報は外部の設定ファイル（application.properties）に記載するため、セキュリティが向上し、環境に応じた変更も容易。
冗長なコード	接続や例外処理などのために冗長なコードが必要。読みにくく、エラーが発生しやすい。	JPA リポジトリインターフェースを使用することで、冗長なコードを大幅に削減。よりシンプルで読みやすい。
例外処理の必要性	DriverManager.getConnection メソッドが接続失敗時にSQLException をスローするため、追加の例外処理コードが必要。	RuntimeException を使用するため、明示的な例外処理が不要になる場合が多いので、コードがシンプル。
リソース管理	開いたデータベース接続を適切に閉じる責任が開発者にある。これを怠るとリソースリークのリスクがある。	リソース管理をフレームワークが行うため、開発者はリソースリークを気にする必要が少ない。

Spring Data JPA は、よりシンプルで安全、そして効率的なデータアクセスを提供することで、データベースプログラミングを容易にします。

演習3.1. Entity と Repository の定義

a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト 3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト 3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

c. repository.EmpRepository に JpaRepository を適切に継承してください。

- テキスト 3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたい Entity 名, Entity の主キーの型名〉

演習 3 Spring Data JPA

演習3.1. Entity とRepository の定義

a. entity.Employee クラスのクラス名の上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順①～②】を参考にしてください。
- Lombok のライブラリを活用してください。

b. 4 つのフィールドの上に適切なアノテーションを付けてください。

- テーブル情報と、テキスト3.2 の【作業手順③】を参考にしてください。
- id(社員番号)には、@GeneratedValue アノテーションを付けてください。
- id 以外の列は、制約がありませんので、@Column の引数は今回不要です。

c. repository.EmpRepository にJpaRepository を適切に継承してください。

- テキスト3.3 の【作業手順④】を参考にしてください。
- 〈紐づけたいEntity 名, Entity の主キーの型名〉